Final Project Report

Design and Implementation of Online Smart Parking Management System

Prepared by: Amirhossein and Parham

June 30, 2026

---

1. Introduction and Project Significance

Main Problem: Finding parking spaces in densely populated cities leads to drivers wasting time, a significant increase in traffic congestion, elevated fuel consumption, and environmental pollution.

Project Environment: This system is designed for public parking lots, large urban multi-story parking structures, and curbside parking spaces.

Necessity of the Internet of Things (IoT): Unlike traditional systems where a driver only realizes a parking lot is full upon physical arrival, an IoT-based system enables real-time monitoring of each parking spot and web-based online reservation. This eliminates vehicle wandering and completely optimizes space management.

2. System Requirements

- **Functional Requirements:** User registration, wallet recharging, real-time viewing of parking space status, firm reservation of an empty spot for a specific time slot, reservation cancellation, and an admin monitoring panel.
- **Qualitative Requirements:** High security in storing identity and financial data (hashing encryption), database stability under concurrent loads, and **instantaneous responsiveness** driven by an Event-Driven Architecture (replacing the traditional, costly Polling mechanism).
- **Network Connectivity:** Sensor nodes installed at each parking spot connect to the network, collecting and sending vehicle presence status data to the central server via an API.

3. Required Equipment and Technologies

3.1. Hardware Component

- **Sensors:** HC-SR04 ultrasonic modules or infrared (IR) sensors.

- **Actuators:** Mini servo motor modules for gate/barrier control.
- **Central Processing Unit:** Microcontrollers such as ESP32 or ESP8266 nodes.

3.2. Software Component

- **Backend:** Python and the Flask framework integrated with the Flask-SocketIO package.
- **Database:** SQLite smart relational database.
- **Frontend and Dashboard:** HTML5, CSS3, JavaScript, and the powerful Three.js library for 3D graphical rendering.

4. System Architecture and Communication Protocols

The overall system architecture is based on a Client-Server and Edge-Server IoT structure.

- **Physical and Network Layer:** Sensor nodes connect to the central gateway using built-in Wi-Fi technology.
- **Data Transfer Protocol (Recent Enhancement):** By completely removing the traditional, high-cost Polling mechanism (consecutive 2-second frontend requests), the system now utilizes **WebSockets**. Any change in sensor status or the database is instantaneously broadcast to the frontend dashboard using the Flask-SocketIO package.

5. State Machine and Behavioral Algorithms

To accurately manage parking traffic, a **Multi-Stage State Machine** has been implemented for vehicles in the system's logic, separating behavioral algorithms into two distinct categories:

- **Pre-Booked (Online Reservations):** Dark blue cars that are guided directly to their specific, pre-allocated parking spot without stopping.
- **On-Site/Walk-In Reservations:** Randomly colored cars that follow a smart 3-stage algorithm: 1. Stopping at the entry line. 2. Sending an instant backend request to reserve the first available database capacity (checking `is_occupied == false`). 3. Dynamic Rerouting toward the 3D coordinates of the newly allocated parking spot.

6. Visual Enhancements and Database Schema Extension

- **Camera-Aware Design:** Obstructive visual elements (like old barrier boxes) have been removed to declutter the digital twin environment. Furthermore, the traditional toll booth window animation has been replaced with a **Floating 3D Indicator** above the booth roof to visually display the on-site transaction process clearly from the top-down camera perspective.

- **Database Schema Extension:** To categorize user reservation methods and prepare data for statistical analysis, a new metadata field named `booking_method` (with values `online` and `on_site`) has been added to the `Reservations` table.

7. Target Users and Value Proposition

- **Target Audience:** Urban drivers, parking lot management companies, and municipal traffic organizations.
- **Value Proposition & Innovation:** Compared to SMS-based systems, digital parking meters, and image-processing systems, our project offers significantly lower hardware costs, immunity to dark lighting conditions, and the unique integration of a comprehensive financial system with an interactive, real-time 3D digital twin.

8. Implementation Feasibility and Workarounds

Operational Evaluation: All backend code, database logic, and frontend 3D dashboards have been successfully implemented. Logic for temporary caching and automatic sensor status release (30 seconds before a reservation starts) resolves issues like unauthorized occupation and Wi-Fi loss.

Alternative Scenario: A native simulator script (`sensor_sim.py`) handles traffic generation and sensor status management, ensuring the platform operates flawlessly even without physical hardware present.

9. Future Work

As part of the project's ongoing development and DevOps roadmap, the following enhancements are proposed:

- **Containerization:** Dockerizing various system components using **Docker Compose** for rapid, reliable, and standardized deployment.
- **Protocol Upgrade:** Upgrading the edge sensor communication from standard HTTP to the industrial **MQTT** protocol using the Eclipse Mosquitto broker to maximize IoT network stability.

10. Initial Cost Estimation

The estimated budget for constructing the prototype is presented in the table below:

Equipment and Service Description	**Estimated Cost (Tomans)**
Central Processing Board (ESP32 NodeMCU)	500,000
Ultrasonic and Infrared Sensors (5 units)	200,000
Actuators and Enclosures (Gate Servo Motor, Wires, Breadboard, Box)	400,000
Communication and Server Hosting Costs	500,000
Total Estimated Budget	**1,600,000 to 2,000,000**